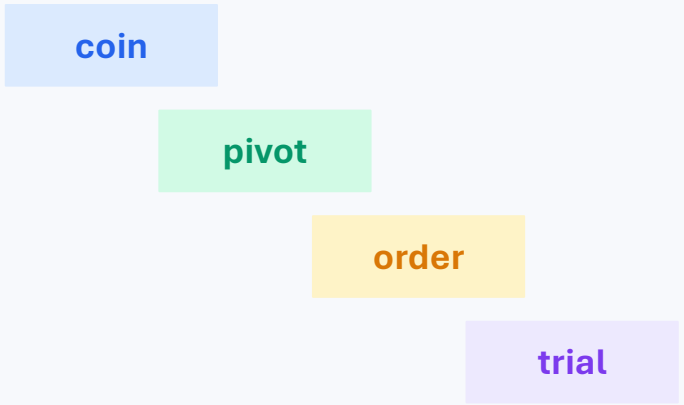
Randomized Algorithms

Introduction to randomization techniques and analysis



ADVANCED ALGORITHMS AND PARALLEL PROGRAMMING
Fabrizio Ferrandi · a.a. 2026–2027
Material adapted from Eric Torng, Michigan State University

Two ways randomness enters the analysis

Probabilistic / average-case analysis

The algorithm is deterministic.
We assume a probability distribution over inputs.

Randomized algorithm

The input may be fixed.
The algorithm makes random choices during execution.

input distribution is assumed

randomness is under our control

Main distinction: passive randomness in the model vs active randomness in the algorithm.

Probabilistic (average-case) analysis

For a fixed input, the deterministic algorithm always behaves the same way.

1 · Assume	2 · Analyze	3 · Check caveats
Choose a probability distribution for the inputs.	Compute the expected quantity of interest over that distribution.	Some inputs may still be much worse. A wrong distribution can be misleading.

Average over inputs, not over runs

Randomized algorithm

For a fixed input, the algorithm may behave differently because of random choices.

Typical randomization techniques

- Randomly permute candidate order
- Randomly select a pivot
- Randomly choose among deterministic strategies

Key guarantees

Works well with high probability on every input. May fail on any fixed input, but only with low probability.

fixed input → **coin tosses** → **different executions**

Key analysis tools

Indicator variables	Linearity of expectation	Recurrence relations
Use X_i to isolate one elementary event. Usually $X = X_1 + X_2 + \cdots + X_n$.	If $X = Y + Z$, then $E[X] = E[Y] + E[Z]$. This does not require independence.	Capture expected behavior recursively, especially for randomized divide-and-conquer algorithms.

Break a random variable into small yes/no events.

Part I

The Hiring Problem

A small example that shows how indicator variables simplify randomized analysis.

Hiring problem: setup

Goal: hire the best applicant seen so far, but each hire is expensive.



Rule

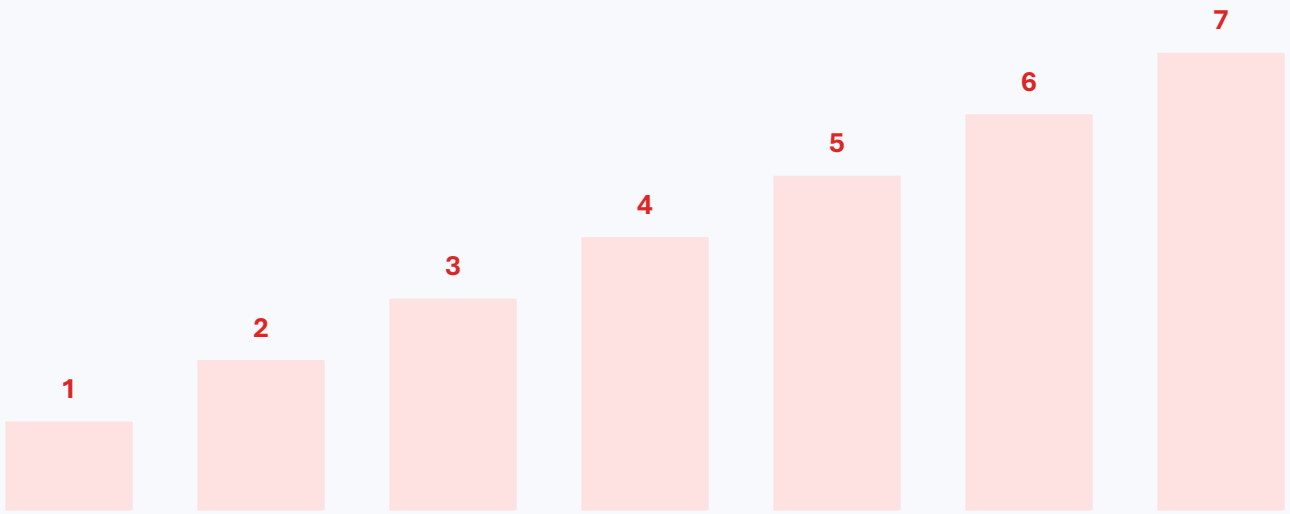
When a new applicant is better than the current employee, fire the current employee and hire the new applicant.

Question

How many expensive hire/fire operations should we expect?

Worst case: increasing quality order

Each applicant is better than everyone seen before.

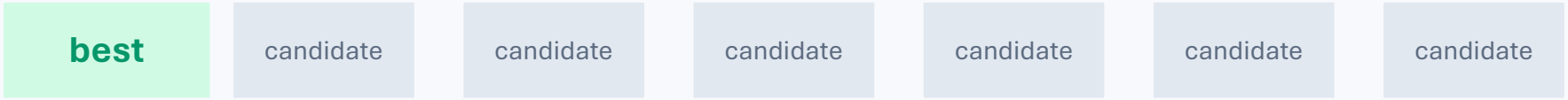


Cost = n hires

The algorithm hires every applicant in turn.

Best case: the best applicant arrives first

The first hire is never replaced.



Cost = 1 hire

Average cost: what distribution?

Input representation	Number of inputs	Common assumption
An input is an ordering of the n applicants.	There are n! possible orderings.	Every ordering is equally likely. Other distributions are possible.

Average cost = expected number of hires

The rest of the analysis computes this expectation.

Model the sample space

Ω = all n! applicant sequences

Pr[s] = 1 / n! for each sequence s

Random variable

X(s) = number of applicants hired for sequence s

Question: what is E[X]?

Use indicator variables

Instead of counting all hires directly, track each applicant separately.

X_i = 1 if applicant i is hired; 0 otherwise

1	0	0	1	0	0	1	0

Each indicator is simple; their sum is the total number of hires.

Why indicators help

$$X = X_1 + X_2 + \cdots + X_n$$

$$E[X_i] = \Pr[X_i = 1]$$

Interpretation

$X_i = 1$ exactly when applicant i is better than all previous i − 1 applicants.

Strategy

Compute $\Pr[X_i = 1]$ for each i, then add the probabilities.

Linearity of expectation lets us add expectations directly.

Example: n = 4, i = 3

Question: in what fraction of all inputs is the 3rd applicant better than the first 2?

pos 1	pos 2	pos 3
any	any	largest of first 3

1234	2134	3124	4123
1243	2143	3142	4132
1324	2314	3214	4213
1342	2341	3241	4231
1423	2413	3412	4312
1432	2431	3421	4321

Among the first 3 positions, the maximum is equally likely to be in any one of them.

8 / 24 = 1 / 3

General probability

For the first i applicants, the best of them is equally likely to appear in any of the i positions.



$$\Pr[X_i = 1] = 1 / i$$

Applicant i is hired only when the best-so-far appears at position i.

Expected number of hires

$$E[X] = E[\Sigma_i X_i] = \Sigma_i E[X_i] = \Sigma_i \Pr[X_i = 1]$$

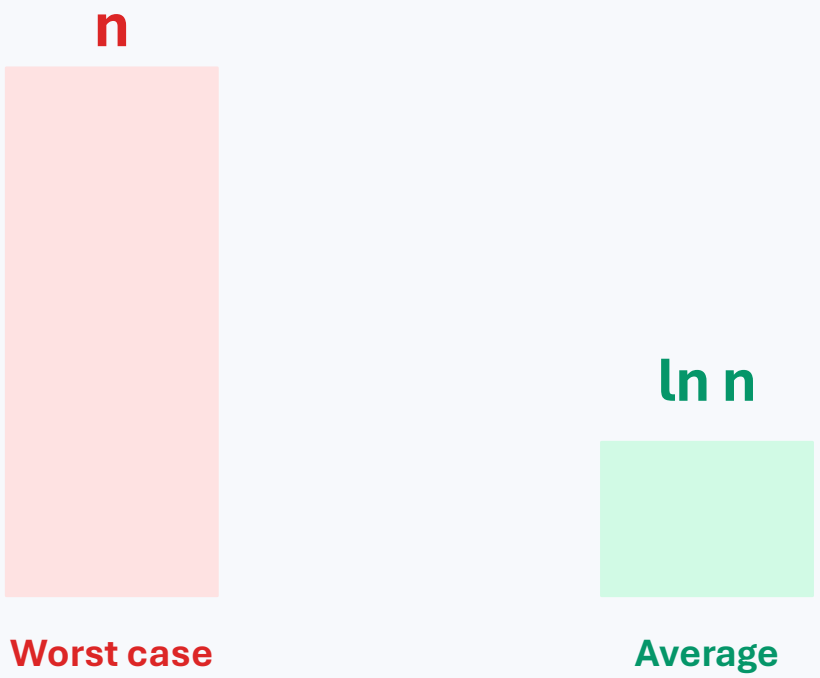
$$= \Sigma_i 1/i = H_n \leq \ln n + 1$$

Result

The average number of hires grows logarithmically, not linearly.

The key lesson

Average cost is much smaller than the worst case — but only if applicant order is random.



If order is unreliable

Do not rely on the headhunter. Randomly permute the applicant list yourself.

From passive to active randomization

Passive probabilistic analysis

Assume inputs follow a distribution.
Risk: the real input distribution may differ.

Active randomized algorithm

Create the distribution yourself by taking random action, such as permuting the input.

Analyze expected behavior on the worst possible input.

The same input can lead to many possible executions.

Randomize the candidate list

Algorithm

1. Get the complete candidate list.
2. Randomly permute the list.
3. Interview in that random order.

Guarantee

Expected number of hirings/firings is $O(\log n)$, regardless of the original input order.

Randomness moves from the input model into the algorithm.

Part II

Las Vegas and Monte Carlo

Two common ways to trade time, correctness, and probability.

Two families of randomized algorithms

Las Vegas

Always returns the correct answer.
Randomness affects the running time.

Monte Carlo

Runs within a bounded time.
Randomness can affect correctness.

Example intuition: randomized sorting vs. randomized min-cut

Monte Carlo error types

For decision problems, the answer is YES or NO.

One-sided error

The algorithm has zero probability of error for at least one possible output.

Two-sided error

There is a non-zero probability of error when the algorithm outputs either YES or NO.

We bound the probability of an incorrect output.

Which type is better?

It depends on the application and the cost of being wrong.

Las Vegas

Error probability is 0.
Efficient if expected running time is polynomial.

Monte Carlo

Running time is bounded.
Efficient if worst-case running time is polynomial and error probability is bounded.

If an incorrect answer is catastrophic, error probability matters more than speed.

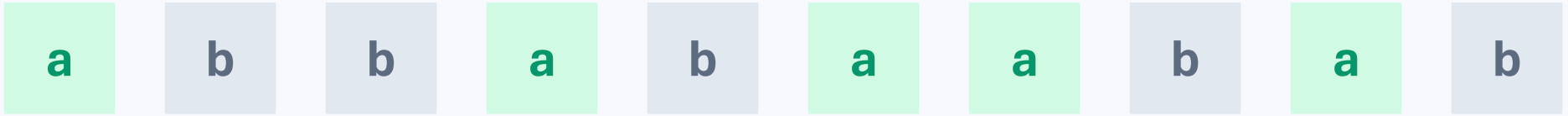
Efficiency

A **Las Vegas** algorithm is an efficient Las Vegas algorithm if on any input its expected running time is bounded by a polynomial function of the input size

A **Monte Carlo** algorithm is an efficient Monte Carlo algorithm if on any input its worst-case running time is bounded by a polynomial function of the input size

Example: find character 'a'

Input: an array of n elements, half 'a' and half 'b'.



Output: find any 'a'

We compare a Las Vegas version and a Monte Carlo version.

Las Vegas version

Repeat until an 'a' is found.

```
findingA_LV(A, n)
repeat
    choose a random index j in {1, …, n}
until A[j] = 'a'
return j
```

Guarantee

Always succeeds.
Running time is random, but E[trials] = 2 because Pr['a'] = 1/2.

Expected time: O(1)

Monte Carlo version

Limit the number of random trials to k.

```
findingA_MC(A, n, k)
for t = 1 to k
    choose a random index j in {1, …, n}
    if A[j] = 'a' then return j
return FAIL
```

Guarantee

Runtime is bounded by k trials.
It may fail if all k samples are 'b'.

Pr[fail] = (1/2)^k

Pr[find 'a'] = 1 – (1/2)^k